Lab# 02

Abstract Classes, Templates, and Arrays in C++

Task 1: Abstract Classes

Scenario: A company wants a system to handle different types of employees. Each employee type has a different way of calculating their monthly salary.

What you will do in this task: Define an abstract class Employee with a pure virtual function calculateSalary(). (5) Create two derived classes, e.g., FullTimeEmployee and PartTimeEmployee, that implement the calculateSalary() function. (7) Demonstrate polymorphism using base class pointers or references to call calculateSalary() for different employee types. (5)

Ensure proper compilation and display the salary output for both types of employees. (3)

Code:

```
#include <iostream>

using namespace std;

class Employee {

public:

   virtual double calculateSalary() = 0;

};

class FullTimeEmployee : public Employee {

private:

   double monthlySalary;

public:

   FullTimeEmployee(double salary) {

      monthlySalary = salary;

   }

   double calculateSalary() {

      return monthlySalary;

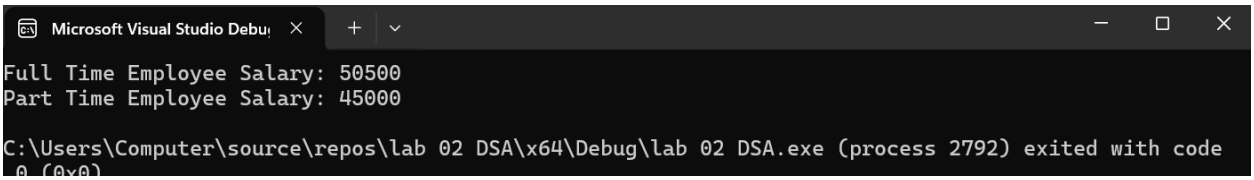
   }

};

class PartTimeEmployee : public Employee {
```

```
private:
    double hoursWorked;
    double hourlyRate;
public:
    PartTimeEmployee(double hours, double rate) {
        hoursWorked = hours;
        hourlyRate = rate;
    }
    double calculateSalary() {
        return hoursWorked * hourlyRate;
    }
};
int main() {
    Employee* emp;
    FullTimeEmployee ft(50500);
    PartTimeEmployee pt(90, 500);
    emp = &ft;
    cout << "Full Time Employee Salary: " << emp->calculateSalary() << endl;
    emp = &pt;
    cout << "Part Time Employee Salary: " << emp->calculateSalary() << endl;
    return 0;
}
```

Output:



Task 2: Template Classes

Scenario: A company wants a generic storage system that can store different types of data like integers, floats, or strings and perform basic operations.

What you will do in this task: Create a template class Storage that supports generic data types. (6)

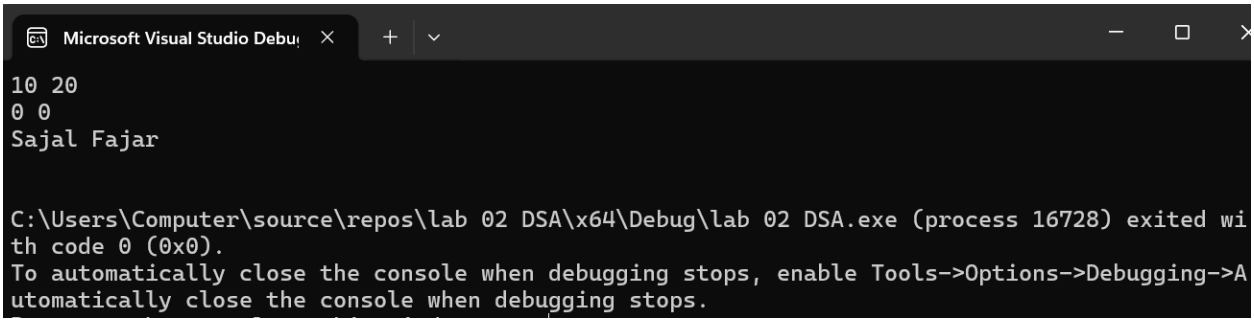
Implement basic operations: insert(), delete(), and display(). (7) Instantiate the template with at least two different data types, e.g., int and string. (5) Ensure correct output and maintain proper code structure. (2)

Code:

```
#include <iostream>
#include <string>
using namespace std;
template <class T>
class Storage {
private:
    T data1, data2;
public:
    void insert(T a, T b) {
        data1 = a;
        data2 = b;
    }
    void deleteItem() {
        data1 = T();
        data2 = T();
    }
    void display() {
        cout << data1 << " " << data2 << endl;
    }
};
int main() {
    Storage<int> s1;
    s1.insert(10, 20);
```

```
    s1.display();
    s1.deleteItem();
    s1.display();
    Storage<string> s2;
    s2.insert("Sajal", "Fajar");
    s2.display();
    s2.deleteItem();
    s2.display();
    return 0;
}
```

Output:



Task 3: Arrays and Their Operations

Scenario: A company needs a system to manage product inventory, where they can add, remove, search, and update products stored in an array or vector.

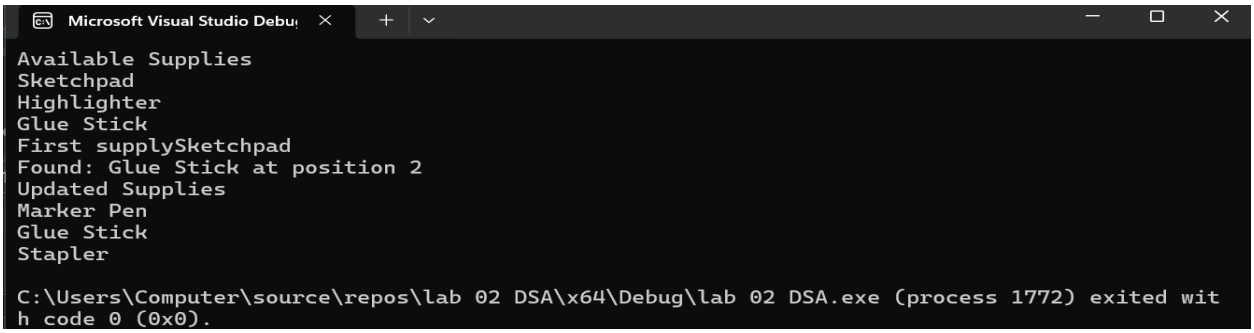
What you will do in this task: Declare and initialize an array or vector to store product information. (5) Perform access and update operations on the array/vector. (5) Implement searching functionality to find a specific product. (5) Implement insertion and deletion of products in the array/vector. (7) Display results clearly to show the current inventory. (3)

Code:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
int main() {
```

```
   vector<string> supplies = { "Sketchpad", "Highlighter", "Glue Stick" };
    cout << "Available Supplies" << endl;
    for (string item : supplies) {
        cout << item << endl;
    }
    cout << "First supply" << supplies[0] << endl;
    supplies[1] = "Marker Pen";
    string target = "Glue Stick";
    bool foundItem = false;
    for (int position = 0; position < supplies.size(); position++) {
        if (supplies[position] == target) {
            cout << "Found: " << supplies[position] << " at position " << position << endl;
            foundItem = true;
            break;
        }
    }
    if (!foundItem) {
        cout << "Item not found in supplies" << endl;
    }
    supplies.push_back("Stapler");
    supplies.erase(supplies.begin());
    cout << "Updated Supplies" << endl;
    for (string item : supplies) {
        cout << item << endl;
    }
    return 0;
}
```

Output:



Task 4: Time Complexity Analysis

Scenario: A company wants to understand the efficiency of different operations in their system by analyzing time complexity. What you will do in this task:

Write a program demonstrating O(1) complexity, e.g., accessing an element in an array. (5) Write a program demonstrating O(n) complexity, e.g., linear search in an array. (5) Write a program demonstrating $O(n^2)$ complexity, e.g., bubble sort or nested loops. (5) Clearly analyze and mention the time complexity in the code comments. (50)

Code:

```
#include <iostream>

using namespace std;

int main() {

   int arr[5] = { 25, 10, 50, 40, 30 };

     cout << arr[2] << endl;

   int search = 40;

      bool found = false;

 for (int i = 0; i < 5; i++) {

       if (arr[i] == search) {

          cout << "Found " << search << " at index " << i << endl;

          found = true;

          break;

       }

   }

   if (!found) {

       cout << "Value not found" << endl;
```

```
    }
    for (int i = 0; i < 5; i++) {
        for (int j = i + 1; j < 5; j++) {
            if (arr[i] > arr[j]) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output:

